

CYS-319-MJEP Web API Security - Practical Practice Questions

Practice Questions with Step-by-Step Solutions. Script names/ports follow the question text; confirm exact URLs, keys and field names against your provided files.

Part A

1.1 Postman CRUD on JSONPlaceholder

Question: In Postman, run four requests against `https://jsonplaceholder.typicode.com/posts`: (i) GET `/posts/1`, (ii) POST `/posts` with body `{"title":"hi","body":"test","userId":1}`, (iii) PUT `/posts/1` with the same body, (iv) DELETE `/posts/1`. For each request, write down the method, endpoint, response body and status code in your journal, with a screenshot of each request-response pair.

Solution (step by step)

1. Open Postman, create a collection 'Practice-A' and set the base URL `https://jsonplaceholder.typicode.com`.
2. **GET** `/posts/1` -> Send. Expected: **200 OK**, body = JSON of post id 1 (userId, id, title, body).
3. **POST** `/posts` -> Body > raw > JSON:
`{"title":"hi","body":"test","userId":1}`
4. Send. Expected: **201 Created**, body echoes the data plus "id": 101 (fake - JSONPlaceholder does not really save).
5. **PUT** `/posts/1` with the same body. Expected: **200 OK**, body = data with id 1.
6. **DELETE** `/posts/1`. Expected: **200 OK**, body = {}.
7. Journal table: Method | Endpoint | Response body | Status (GET 200, POST 201, PUT 200, DELETE 200). Take a screenshot of each request/response pair.

Viva: 201 means a resource was created; PUT replaces the whole resource, PATCH changes part of it; DELETE is idempotent.

1.2 Flask Student REST API - five operations

Question: Run the provided `p2_server.py`, a Flask REST API for a 'Student' resource on port 5000. In Postman, perform all five operations - GET `/students`, GET `/students/<id>`, POST `/students`, PUT `/students/<id>`, DELETE `/students/<id>` - and show the status code returned by each (200/201/404 as applicable).

Solution (step by step)

1. Run: `python p2_server.py` (Flask on `http://127.0.0.1:5000`). Keep the terminal open.
2. **GET** `/students` -> 200 + list of students.
3. **GET** `/students/1` -> 200 + one student. Try `/students/999` -> **404** Not Found.
4. **POST** `/students`, Body raw JSON, e.g. `{"name":"Amit","marks":80}` -> **201 Created**.
5. **PUT** `/students/1` with changed JSON -> **200 OK** (404 if the id does not exist).
6. **DELETE** `/students/1` -> **200** (or 204); repeat the DELETE -> **404**.
7. Screenshot each call with its status code and fill a table: Operation | URL | Status.

1.3 Nginx rate limiting with client script

Question: Start `p3_backend.py` (port 5000), then start Nginx with the provided `nginx.conf`, which proxies port 8080 to the backend and enforces `limit_req_zone` (1 request/sec, burst=3). Run `p3_client.py`, which fires 20 rapid GET requests at `http://127.0.0.1:8080/students`. Show that the first few return 200 and the rest return 503, and confirm in `access.log` that the blocked ones were stopped by Nginx.

Solution (step by step)

1. Terminal 1: python p3_backend.py (Flask on port 5000).
2. Terminal 2: start Nginx with the given config, e.g. nginx -c <full path>\nginx.conf (Windows: from the nginx folder, start nginx). Key config lines:

```
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=1r/s;
server { listen 8080;
    location / { limit_req zone=mylimit burst=3;
        limit_req_status 503;
        proxy_pass http://127.0.0.1:5000; } }
```
3. Run python p3_client.py - it fires 20 quick GETs at http://127.0.0.1:8080/students.
4. Observe: the first ~4 requests (1 allowed + burst 3) return **200**; the rest return **503** Service Temporarily Unavailable.
5. Open Nginx logs/access.log: lines with " 200 are passed, lines with " 503 are blocked. The Flask console shows only the 200 requests - proof Nginx stopped the others.

1.4 Quota gateway monitoring

Question: Start p4_gateway.py, run p4_simulate_clients.py (sends 8 requests as client_A and 3 as client_B), then run p4_monitor.py, which reads the gateway's /stats endpoint and access.log. Report how many requests each client made, which client is currently blocked, and for how many more seconds.

Solution (step by step)

1. Start gateway: python p4_gateway.py (quota 5 req/15 s per client, cooldown 20 s).
2. Run python p4_simulate_clients.py -> client_A sends 8 requests (first 5 = 200, then 403 'quota exceeded'), client_B sends 3 (all 200).
3. Immediately run python p4_monitor.py (reads GET /stats and access.log).
4. Report from its output: client_A made **8** requests, client_B made **3**; **client_A is blocked**, with remaining time = 20 s minus time since the block began (e.g. ~15 s); client_B is not blocked.

Run the monitor again after ~20 s to show client_A is unblocked automatically.

1.5 Juice Shop - Password Hash Leak

Question: With Burp Suite's proxy running, log in to OWASP Juice Shop and complete the 'Password Hash Leak' challenge (Sensitive Data Exposure). Using Burp's HTTP history, find the exact API response that leaks the logged-in user's password hash, and explain in one line why an API should never return a password hash field, even to the user it belongs to.

Solution (step by step)

1. Install/run OWASP Juice Shop (e.g. docker run -p 3000:3000 bkimminich/juice-shop). Open Burp Suite > Proxy > Intercept off; open Burp's browser (or set the browser proxy to 127.0.0.1:8080).
2. Browse to http://localhost:3000, register a user and log in.
3. Burp > Proxy > HTTP history: look for the API call made after login, e.g. GET /rest/user/whoami or the user-details request (/api/Users/<id> /rest/user/authentication-details).
4. Open the response: it contains "password": "0192023a7bbd73250516f069df18b500" (an MD5 hash). That is the exact leaking response; copy it into the journal and screenshot it.
5. Check the Score Board (http://localhost:3000/#/score-board) - 'Password Hash Leak' shows solved.

One-line reason: a hash can be cracked offline (MD5 is weak, rainbow tables), so the server should never send it; clients have no need for it, and returning it exposes it to XSS, proxies and logs.

1.6 SQL injection login bypass

Question: Start p6_vuln.py (port 5000) and p6_secure.py (port 5001), or run p6_login_client.py. Send the payload username=admin' -- to /login on both. Show the vulnerable API returns {"login":"SUCCESS"}

(authentication bypassed) while the secure API returns {"login":"REJECTED"} because its regex rejects any non-alphanumeric character. State the one-line fix that stops the bypass.

Solution (step by step)

1. Start both: python p6_vuln.py (port 5000) and python p6_secure.py (port 5001), or just python p6_login_client.py.

2. Vulnerable API builds the query by string concatenation:

```
q = "SELECT * FROM users WHERE username='" + u + "' AND password='" + p + "'"
```

3. Send payload username=admin' -- to /login on port 5000, e.g. curl -X POST "http://127.0.0.1:5000/login" -d "username=admin' --" -d "password=x" (or in Postman Body x-www-form-urlencoded).

4. The query becomes ... WHERE username='admin' --' AND password='x'; -- comments out the password check -> response {"login":"**SUCCESS**"} (bypass).

5. Send the same payload to port 5001: the regex (e.g. ^[A-Za-z0-9]+\$) rejects the quote and spaces -> {"login":"**REJECTED**"}.

6. **One-line fix:** use parameterized queries: cursor.execute("SELECT * FROM users WHERE username=? AND password=?", (u, p)) (plus input validation).

1.7 Basic Auth, API Key and JWT

Question: Start p7_auth_all.py (or run p7_all_client.py). For EACH of its three schemes - Basic Auth (/basic/students), API Key (/apikey/students), and JWT (/login then /jwt/students) - show one request with missing/incorrect credentials (401/403) and one with correct credentials (200 with data).

Solution (step by step)

1. Run python p7_auth_all.py (or python p7_all_client.py which does all calls).

2. **Basic Auth** GET /basic/students: no/wrong credentials -> **401**; Postman Authorization > Basic Auth (correct user/pass) -> **200** + data.

3. **API Key** GET /apikey/students: missing/wrong header X-API-Key -> **401/403**; correct key -> **200**.

4. **JWT:** POST /login with valid username/password -> returns token. GET /jwt/students without token or with a bad one -> **401**; with header Authorization: Bearer <token> -> **200**.

5. Tabulate: Scheme | Request | Status for the 6 requests and add screenshots.

Part B

2.1 Duplicate-id bug in Student API

Question: Run the provided p2_server.py (Flask 'Student' API, port 5000). In Postman: POST two new students, DELETE the first of the two, then POST a third student. Show that the new student is given the SAME id as a student that already exists (a duplicate-id bug caused by computing $\text{id} = \text{len}(\text{list}) + 1$) then edit p2_server.py to fix it and re-run the same steps to show ids are now unique.

Solution (step by step)

1. Run python p2_server.py.
2. Postman: POST /students (student A) -> id 3 (say, if 2 exist already: ids 1,2 then A=3), POST /students (student B) -> id 4.
3. DELETE the first of the two new ones (A, id 3). The list now has length 3 (ids 1,2,4).
4. POST a third student C: the buggy code computes $\text{id} = \text{len}(\text{students}) + 1 = 4$ -> C gets id 4, the SAME as B. GET /students shows two records with id 4 = duplicate-id bug.
5. Fix in p2_server.py - use a global counter (or $\text{max id} + 1$):

```
# bug
new_id = len(students) + 1
# fix 1
next_id = 1          # global, defined once
new_id = next_id; next_id += 1
# fix 2
new_id = max([s['id'] for s in students], default=0) + 1
```

6. Restart the server and repeat the steps: ids are now 3, 4, then 5 - all unique. Show before/after screenshots.

2.2 Nginx gateway - allowed vs blocked

Question: Start p3_backend.py (port 5000) and start Nginx with the given nginx.conf (gateway on port 8080, rate limit 1 request/sec, burst 3). Send a mix of slow and rapid requests to <http://127.0.0.1:8080/students>. Open access.log, point to one allowed (200) line and one blocked (503) line, and explain why Nginx blocks it before the request ever reaches the Flask backend.

Solution (step by step)

1. Start python p3_backend.py (5000) and Nginx with the given nginx.conf (listen 8080, limit_req_zone ... rate=1r/s, burst=3).
2. Send slow requests (one per 2 s) to <http://127.0.0.1:8080/students> -> all **200**. Then send rapid ones (hold Send in Postman runner / curl loop) -> some **503**.
3. Open logs/access.log: point to one line ending " 200 ... (allowed) and one ending " 503 ... (blocked).
4. Explanation: Nginx sits in front as a reverse proxy/gateway. The limit_req module counts requests per client IP in shared memory *before* proxy_pass runs; when the rate exceeds 1 r/s + burst 3 it immediately returns 503 itself, so the request is never forwarded and Flask never sees it (Flask log has no entry). This protects the backend from DoS/brute force.

2.3 Per-client quota and cooldown

Question: Start p4_gateway.py (quota = 5 requests/15s per client, cooldown = 20s). Send requests with header X-Client-Id: client_A until it is blocked (403 'quota exceeded'), while client_B (fewer requests) keeps getting 200. Wait out the cooldown and show client_A is automatically allowed again without restarting the server.

Solution (step by step)

1. Run python p4_gateway.py (quota 5 requests/15 s per client, cooldown 20 s).

2. Postman GET to the gateway URL with header X-Client-Id: client_A; send 6+ times quickly: requests 1-5 -> **200**, 6th -> **403 'quota exceeded'**.
3. Meanwhile use X-Client-Id: client_B and send 2-3 requests -> always **200** (quota tracked per client).
4. Wait 20 s (cooldown), send as client_A again -> **200**. The gateway unblocked it automatically, no restart needed.
5. Screenshots: 200s, the 403, client_B 200, and client_A 200 after cooldown.

2.4 Juice Shop - Broken Object Level Authorization

Question: With Burp Suite's proxy running and your browser routed through it, log in to OWASP Juice Shop and complete EITHER the 'View Basket' OR 'Forged Feedback' challenge (both are Broken Object Level Authorization) using Burp Repeater by changing the basket id or the UserId field in the request. Show the original request, your modified request, and the response that proves the Score Board marks the challenge solved.

Solution (step by step)

1. Start Juice Shop (docker run -p 3000:3000 bkimminich/juice-shop); Burp Proxy on, Intercept ON, browser via Burp. Register and log in as user A.
2. **Option 1 - View Basket:** open 'Your Basket'. In Burp intercept find GET /rest/basket/6 (your own id) with the Bearer token; Send to Repeater (Ctrl+R).
3. In Repeater change the id to another user's basket, e.g. /rest/basket/1 (or 2) -> Send: **200** and the other user's basket items are returned (BOLA).
4. **Option 2 - Forged Feedback:** submit feedback, capture POST /api/Feedbacks, send to Repeater, add/change "UserId": 2 in the JSON body, Send -> feedback is created in another user's name.
5. Open http://localhost:3000/#/score-board -> the challenge is marked solved (green).
6. Show: original request, modified request, response, Score Board screenshot.

Cause: the server checks only that you are logged in, not that the object (basket/UserId) belongs to you.

2.5 HTTP Parameter Pollution on /transfer

Question: Start p6_transfer_vuln.py (port 5003) and p6_transfer_secure.py (port 5004), or run p6_transfer_client.py. Send GET /transfer?amount=100&amount=99999 to both. Show the vulnerable API validates 100 but actually transfers 99999, because validation and the transfer step read different occurrences of the repeated 'amount' parameter, while the secure API rejects the request for containing a duplicate parameter.

Solution (step by step)

1. Run python p6_transfer_vuln.py (5003) and python p6_transfer_secure.py (5004) (or p6_transfer_client.py).
2. Request: curl "http://127.0.0.1:5003/transfer?amount=100&amount=99999" (or Postman with two 'amount' params).
3. Vulnerable API: the validation step reads the **first** value (100, within limit -> passes) but the transfer step reads the **last** value (99999) -> transfers **99999**. This is HTTP Parameter Pollution.
4. Same request to port 5004: the secure API checks the parameter occurs only once (e.g. len(request.args.getlist('amount')) != 1 -> 400) and rejects it.
5. Show both responses side by side with screenshots.

2.6 JWT login and expiry

Question: Start p7_jwt.py (or run p7_jwt_client.py). Log in with POST /login (admin/admin123) to get a JWT (expires in 30s), decode it on jwt.io and show its header/payload/signature. Call GET /students with the token

immediately (200 OK), then again after waiting 31 seconds (401 'token expired').

Solution (step by step)

1. Run `python p7_jwt.py` (or `p7_jwt_client.py`).
2. Postman: POST /login, Body JSON {"username":"admin","password":"admin123"} -> response contains a token (valid 30 s).
3. Paste the token on <https://jwt.io>: **Header** = {alg: HS256, typ: JWT}; **Payload** = {user/sub, exp (expiry time)}; **Signature** = HMACSHA256(base64(header).base64(payload), secret).
4. Immediately GET /students with Authorization > Bearer Token = token -> **200 OK** + data.
5. Wait 31 seconds and resend the same request -> **401** 'token expired'. Log in again for a new token.

2.7 HTTPS and Burp response tampering

Question: Run `p8_gen_cert.py` once, then start `p8_https_app.py` (HTTPS on port 5443). Trust Burp's CA in your browser, proxy your HTTPS traffic through Burp, and in Repeater tamper with the 'marks' field of the GET /students/1 response. State what this shows about what TLS protects (data in transit) and what it does NOT protect (the endpoint itself, once a trusted proxy decrypts the traffic).

Solution (step by step)

1. Run `python p8_gen_cert.py` once (creates `cert.pem/key.pem`), then `python p8_https_app.py` (`https://127.0.0.1:5443`).
2. Burp: Proxy > Options listener 127.0.0.1:8080. Export Burp CA (browse to `http://burpsuite` -> CA Certificate) and import it into the browser's trusted authorities. Route the browser through Burp.
3. Open `https://127.0.0.1:5443/students/1` in the browser; in Burp HTTP history send the request to Repeater.
4. In Repeater, Send, then edit the response (use Intercept > 'Response to this request' and change "marks": 80 to "marks": 100) - the browser shows the tampered value.
5. **Conclusion:** TLS protects data *in transit* (confidentiality and integrity between client and server; an outsider cannot read or alter it). It does NOT protect the endpoint itself: once a proxy the client trusts (Burp CA) decrypts the traffic, the user/attacker can read and modify requests and responses. So the server must still do authentication, authorization and input validation - TLS is not a substitute.